0. Marco Teórico: Naive Bayes
1. Carga de Librerías y Datos
2. Preprocesamiento de Datos
3. Modelo de Regresión con Naive Bayes (Actividad 1)
4. Comparación con Modelos de Regresión Anteriores (Actividad 3)
5. Modelo de Clasificación con Naive Bayes (Actividad 4)
6. Predicciones y Evaluación en Test (Actividad 5)
7. Análisis de la Matriz de Confusión (Actividad 6)
8. Análisis de Variables Más Importantes
9. Resumen y Conclusiones del Pasaporte

Laboratorio 5 – Naive Bayes: Predicción y Clasificación de Precios Airbnb Austin

Alejandro Antón, Juan Fernando, Fernando Mendoza

2026-03-27

---

0. Marco Teórico: Naive Bayes

0.1 Fundamento Probabilístico

El clasificador Naive Bayes parte del Teorema de Bayes, que relaciona las probabilidades condicionales y marginales de un evento:

$$P(C_k \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid C_k)\, P(C_k)}{P(\mathbf{x})}$$

En esta expresión, $P(C_k \mid \mathbf{x})$ denota la probabilidad posterior de que la observación pertenezca a la clase C_k una vez conocido el vector de predictores $\mathbf{x} = (x_1, \ldots, x_p)$. El término $P(\mathbf{x} \mid C_k)$ corresponde a la verosimilitud, $P(C_k)$ a la probabilidad a priori de la clase, y $P(\mathbf{x})$ a la evidencia marginal, que al ser constante para todas las clases resulta irrelevante en la decisión de clasificación.

0.2 Supuesto de Independencia Condicional

El adjetivo *naive* hace referencia al supuesto de que, condicionado a la clase, los predictores son mutuamente independientes:

$$P(\mathbf{x} \mid C_k) = \prod_{j=1}^{p} P(x_j \mid C_k)$$

Gracias a esta simplificación, en lugar de estimar una distribución conjunta p-dimensional basta con estimar p distribuciones univariadas por clase. Domingos y Pazzani (1997) mostraron que, pese a que el supuesto rara vez se cumple de manera estricta, la regla de clasificación resultante mantiene un desempeño aceptable siempre que el ordenamiento relativo de las probabilidades posteriores se preserve, aunque sus magnitudes absolutas estén sesgadas. En el contexto de Airbnb, variables como `bedrooms`, `beds` y `accommodates` presentan correlaciones altas, por lo que conviene tener en cuenta esta limitación al interpretar las probabilidades estimadas.

0.3 Regla de Decisión MAP

La asignación de clase se realiza mediante el criterio de *Maximum A Posteriori*:

$$\hat{C} = \arg\max_{k} \; P(C_k) \prod_{j=1}^{p} P(x_j \mid C_k)$$

0.4 Suavizado de Laplace

Cuando una combinación predictor–clase no se observa en el conjunto de entrenamiento, la verosimilitud correspondiente es cero y anula el producto completo. El suavizado de Laplace resuelve este problema añadiendo un conteo ficticio α (habitualmente $\alpha = 1$) a cada celda de la tabla de frecuencias:

$$\hat{P}(x_j = v \mid C_k) = \frac{n_{kv} + \alpha}{n_k + \alpha \cdot |V_j|}$$

donde n_{kv} es la frecuencia de la combinación (v, C_k), n_k el total de observaciones de la clase k, y $|V_j|$ el número de valores distintos del predictor j.

0.5 Extensión a Regresión

Naive Bayes es, por construcción, un clasificador. No obstante, es posible derivar una predicción numérica a partir de las probabilidades posteriores calculando la esperanza condicional del precio:

$$\hat{y} = \sum_{k=1}^{K} P(C_k \mid \mathbf{x}) \cdot \bar{y}_k$$

donde $\bar{y}_k$ representa el precio promedio observado en el entrenamiento para la clase k. De este modo, la predicción continua es un promedio ponderado por la confianza del modelo en cada categoría. Cabe señalar que esta aproximación comprime las predicciones al rango de los promedios por clase, perdiendo la capacidad de discriminar precios dentro de una misma categoría.

---

1. Carga de Librerías y Datos

```
library(tidyverse)
library(e1071)
library(caret)
library(ggplot2)
library(knitr)
library(kableExtra)
library(scales)
library(rpart)
library(rpart.plot)
library(randomForest)
library(Metrics)

# Cargar datos (mismo archivo que Labs 3 y 4)
load("listings.RData")
df <- listings

glimpse(df)
```

```
## Rows: 5,000
## Columns: 24
## $ id                      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,…
## $ scrape_id               <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,…
## $ host_id                 <int> 415, 463, 179, 14, 195, 426, 306, 118, 299, 22…
## $ price                   <chr> "$129", "$328", "$187", "$91", "$253", "$336",…
## $ accommodates            <int> 3, 3, 4, 7, 7, 8, 4, 9, 8, 3, 10, 2, 6, 5, 7, …
## $ bedrooms                <int> 2, 2, 2, 3, 2, 2, 2, 3, 1, 2, 1, 5, 4, 1, 1, 1…
## $ bathrooms               <dbl> 2.5, 3.3, 1.7, 1.8, 1.7, 1.2, 1.7, 3.4, 3.7, 1…
## $ beds                    <int> 1, 5, 5, 5, 2, 5, 8, 6, 5, 2, 3, 6, 3, 1, 2, 3…
## $ minimum_nights          <int> 24, 16, 8, 5, 22, 17, 10, 14, 14, 11, 14, 7, 2…
## $ maximum_nights          <int> 361, 49, 233, 55, 158, 236, 255, 293, 253, 206…
## $ minimum_minimum_nights  <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
## $ maximum_minimum_nights  <dbl> 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30…
## $ minimum_maximum_nights  <dbl> 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30…
## $ maximum_maximum_nights  <dbl> 365, 365, 365, 365, 365, 365, 365, 365, 365, 3…
## $ minimum_nights_avg_ntm  <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2…
## $ maximum_nights_avg_ntm  <dbl> 365, 365, 365, 365, 365, 365, 365, 365, 365, 3…
## $ availability_30         <int> 24, 19, 14, 7, 13, 0, 4, 21, 7, 2, 14, 17, 13,…
## $ availability_60         <int> 40, 31, 53, 39, 44, 28, 11, 33, 51, 59, 15, 10…
## $ availability_90         <int> 1, 22, 8, 71, 74, 49, 30, 86, 11, 19, 88, 61, …
## $ availability_365        <int> 303, 65, 60, 109, 119, 174, 47, 79, 279, 86, 3…
## $ estimated_revenue_l365d <dbl> 45448.818, 11295.995, 23268.049, 34797.330, 34…
## $ number_of_reviews       <int> 171, 185, 89, 185, 138, 32, 155, 200, 165, 62,…
## $ review_scores_rating    <dbl> 3.091646, 4.519871, 4.858564, 3.382514, 3.9238…
## $ room_type               <chr> "Shared room", "Entire home/apt", "Entire home…
```

2. Preprocesamiento de Datos

El pipeline de limpieza que se aplica a continuación es idéntico al empleado en los Laboratorios 3 y 4. Mantener las mismas transformaciones y la misma semilla de partición es un requisito necesario para que las métricas de los distintos algoritmos sean directamente comparables.

2.1 Limpieza

```
df_clean <- df %>%
  select(
    -id, -scrape_id, -host_id,
    -minimum_minimum_nights, -maximum_minimum_nights,
    -minimum_maximum_nights, -maximum_maximum_nights,
    -minimum_nights_avg_ntm, -maximum_nights_avg_ntm,
    -availability_30, -availability_60, -availability_90,
    -estimated_revenue_l365d
  ) %>%
  drop_na(price, accommodates, bedrooms, bathrooms, beds) %>%
  mutate(price = as.numeric(gsub("[\\$,]", "", price))) %>%
  filter(!is.na(price) & price > 0)

cat("Dimensiones del dataset limpio:", dim(df_clean), "\n")
```

```
## Dimensiones del dataset limpio: 5000 11
```

Se excluyen identificadores sin valor predictivo, agregados redundantes de noches y disponibilidad, y la variable de ingresos estimados por riesgo de *data leakage*. Los registros con valores faltantes en variables estructurales (precio, capacidad, habitaciones, baños, camas) se descartan, y la columna `price` se convierte a formato numérico tras eliminar el símbolo de dólar y las comas.

2.2 División Train / Test

```
set.seed(123)
n <- nrow(df_clean)
train_index <- sample(1:n, size = 0.7 * n)

train <- df_clean[train_index, ]
test  <- df_clean[-train_index, ]

cat("Observaciones de entrenamiento:", nrow(train), "\n")
```

```
## Observaciones de entrenamiento: 3500
```

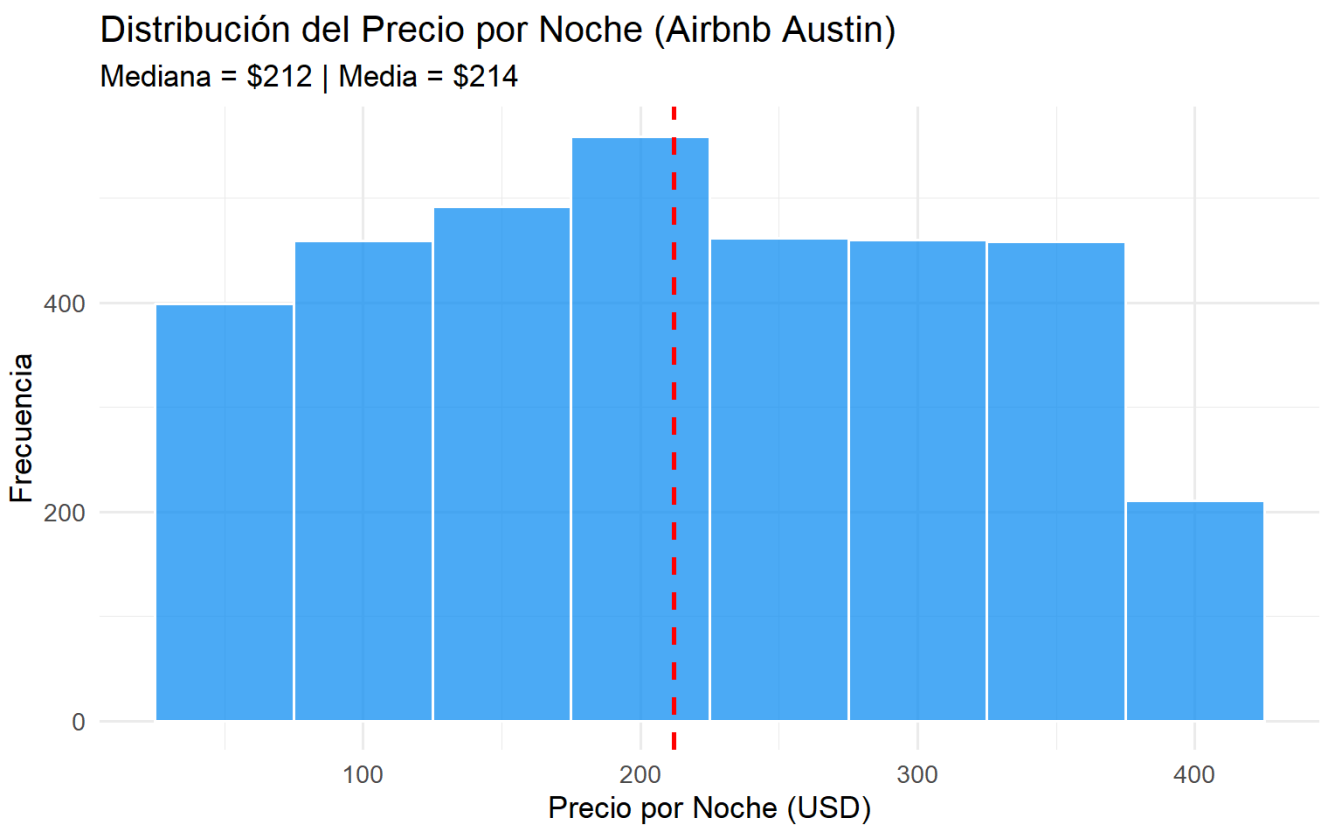
```
cat("Observaciones de prueba:       ", nrow(test), "\n")
```

```
## Observaciones de prueba:        1500
```

La semilla y el método de muestreo (`set.seed(123)` con `sample`) reproducen exactamente la misma partición 70/30 de las entregas previas.

2.3 Distribución del Precio

```
train %>%
  filter(price < 2000) %>%
  ggplot(aes(x = price)) +
  geom_histogram(binwidth = 50, fill = "#2196F3", color = "white", alpha = 0.8) +
  geom_vline(xintercept = median(train$price), color = "red",
             linetype = "dashed", size = 1) +
  labs(
    title    = "Distribución del Precio por Noche (Airbnb Austin)",
    subtitle = paste0("Mediana = $", round(median(train$price), 0),
                      " | Media = $", round(mean(train$price), 0)),
    x = "Precio por Noche (USD)", y = "Frecuencia"
  ) +
  theme_minimal(base_size = 13)
```



La distribución presenta asimetría positiva pronunciada: la masa se concentra entre $50 y $300, mientras que la cola derecha se extiende hasta varios miles de dólares por las propiedades de lujo. Esta forma condiciona la elección de percentiles —en lugar de intervalos equidistantes— como criterio de discretización.

2.4 Creación de la Variable Categórica de Precio

Se retoman las tres categorías definidas en el Laboratorio 4 (Económica, Intermedia y Cara), cuyos límites corresponden a los percentiles 33 y 66 de la distribución completa.

```
p33 <- quantile(df_clean$price, probs = 0.33)
p66 <- quantile(df_clean$price, probs = 0.66)

cat("Percentil 33 (límite Económica-Intermedia): $", p33, "\n")
```

```
## Percentil 33 (límite Económica-Intermedia): $ 152
```

```
cat("Percentil 66 (límite Intermedia-Cara):      $", p66, "\n")
```

```
## Percentil 66 (límite Intermedia-Cara):      $ 271
```

```
asignar_categoria <- function(precio, umbral_bajo, umbral_alto) {
  case_when(
    precio <= umbral_bajo  ~ "Economica",
    precio <= umbral_alto  ~ "Intermedia",
    TRUE                   ~ "Cara"
  )
}

train$categoria_precio <- factor(
  asignar_categoria(train$price, p33, p66),
  levels = c("Economica", "Intermedia", "Cara")
)
test$categoria_precio <- factor(
  asignar_categoria(test$price, p33, p66),
  levels = c("Economica", "Intermedia", "Cara")
)

cat("\nDistribución en entrenamiento (%):\n")
```

```
##
## Distribución en entrenamiento (%):
```

```
print(round(prop.table(table(train$categoria_precio)) * 100, 1))
```

```
##
##  Economica Intermedia       Cara
##       32.8       33.7       33.5
```

```
cat("\nDistribución en prueba (%):\n")
```

```
##
## Distribución en prueba (%):
```

```
print(round(prop.table(table(test$categoria_precio)) * 100, 1))
```

```
##
##  Economica Intermedia       Cara
##       34.3       31.3       34.5
```

Los terciles producen grupos de tamaño aproximadamente igual (~33 % cada uno), lo cual evita desbalances que sesguen el entrenamiento del clasificador. Un modelo que siempre prediga la clase mayoritaria obtendrá, como máximo, alrededor del 34 % de exactitud; esta cifra constituye la línea base contra la cual se evalúa cualquier modelo.

2.5 Preparación de Variables para Naive Bayes

El algoritmo requiere que los predictores sean categóricos o, en su defecto, que asuman una distribución paramétrica conocida. Se optó por discretizar las variables numéricas en intervalos con interpretación sustantiva, lo cual además facilita la estimación de las tablas de frecuencias condicionadas sin asumir normalidad.

```
preparar_nb <- function(datos) {
  datos %>%
    mutate(
      accommodates_cat = cut(accommodates,
                             breaks = c(0, 2, 4, 6, Inf),
                             labels = c("1-2", "3-4", "5-6", "7+"),
                             right = TRUE),
      bedrooms_cat = cut(bedrooms,
                         breaks = c(-Inf, 0, 1, 2, 3, Inf),
                         labels = c("0", "1", "2", "3", "4+")),
      beds_cat = cut(beds,
                     breaks = c(-Inf, 1, 2, 3, Inf),
                     labels = c("1", "2", "3", "4+")),
      bathrooms_cat = cut(bathrooms,
                          breaks = c(-Inf, 1, 1.5, 2, Inf),
                          labels = c("1", "1.5", "2", "2+")),
      min_nights_cat = cut(minimum_nights,
                           breaks = c(0, 1, 3, 7, 30, Inf),
                           labels = c("1", "2-3", "4-7", "8-30", "31+"),
                           right = TRUE),
      room_type = as.factor(room_type)
    )
}

train_nb <- preparar_nb(train) %>% drop_na(accommodates_cat, bedrooms_cat,
                                           beds_cat, bathrooms_cat, min_nights_cat)
test_nb  <- preparar_nb(test) %>% drop_na(accommodates_cat, bedrooms_cat,
                                          beds_cat, bathrooms_cat, min_nights_cat)

cat("Observaciones NB — Train:", nrow(train_nb), "| Test:", nrow(test_nb), "\n")
```

```
## Observaciones NB — Train: 3500 | Test: 1500
```

Los puntos de corte se definieron considerando la distribución marginal de cada variable y su relación conocida con el precio (por ejemplo, la capacidad de 7 o más huéspedes marca el umbral a partir del cual la mayoría de propiedades son de tipo “Entire home/apt” de gran tamaño). Se eliminan las pocas filas que presentan valores faltantes tras la discretización.

---

3. Modelo de Regresión con Naive Bayes (Actividad 1)

3.1 Estrategia

Siguiendo el procedimiento descrito en la Sección 0.5, se entrena un modelo de clasificación sobre las tres categorías de precio y luego se obtienen las probabilidades posteriores para calcular una predicción numérica mediante esperanza condicional.

```
formula_nb <- categoria_precio ~ room_type + accommodates_cat + bathrooms_cat +
  bedrooms_cat + beds_cat + min_nights_cat

modelo_nb <- naiveBayes(formula_nb, data = train_nb, laplace = 1)

precio_por_cat <- train_nb %>%
  group_by(categoria_precio) %>%
  summarise(precio_medio = mean(price, na.rm = TRUE)) %>%
  arrange(categoria_precio)

cat("Precio promedio por categoría (entrenamiento):\n")
```

```
## Precio promedio por categoría (entrenamiento):
```

```
print(precio_por_cat)
```

```
## # A tibble: 3 × 2
##   categoria_precio precio_medio
##   <fct>                   <dbl>
## 1 Economica                93.5
## 2 Intermedia              211.
## 3 Cara                    335.
```

```
prob_test <- predict(modelo_nb, newdata = test_nb, type = "raw")

medias_vec <- precio_por_cat$precio_medio
pred_nb_reg <- as.numeric(prob_test %*% medias_vec)
```

El vector `medias_vec` contiene los tres precios promedio que sirven como centroides de cada categoría. La predicción de cada observación es, por tanto, una combinación convexa de estos tres valores ponderada por las probabilidades posteriores que el modelo asigna.

3.2 Resultados del Modelo de Regresión NB (Actividad 2)

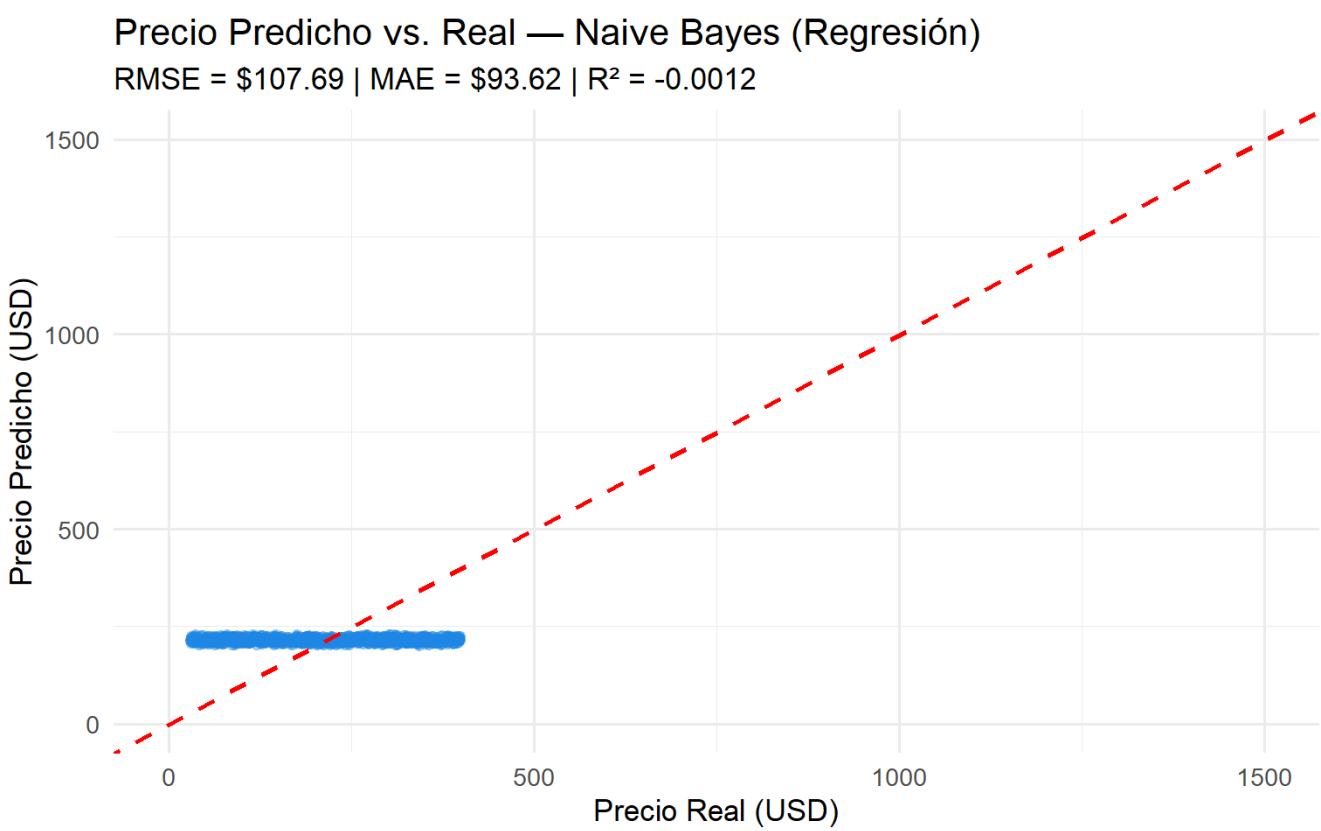
```
actual_nb <- test_nb$price

rmse_nb <- sqrt(mean((actual_nb - pred_nb_reg)^2))
mae_nb  <- mean(abs(actual_nb - pred_nb_reg))
r2_nb   <- 1 - sum((actual_nb - pred_nb_reg)^2) / sum((actual_nb - mean(actual_nb))^
         2)
mse_nb  <- mean((actual_nb - pred_nb_reg)^2)

metricas_nb_reg <- data.frame(
  Métrica = c("MSE", "RMSE", "MAE", "R²"),
  Valor   = c(round(mse_nb, 2), round(rmse_nb, 2), round(mae_nb, 2), round(r2_nb,
         4))
)

kable(metricas_nb_reg,
      caption = "Métricas de Regresión — Naive Bayes") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Métricas de Regresión — Naive Bayes

Métrica	Valor
MSE	11597.1600
RMSE	107.6900
MAE	93.6200
R^2	-0.0012

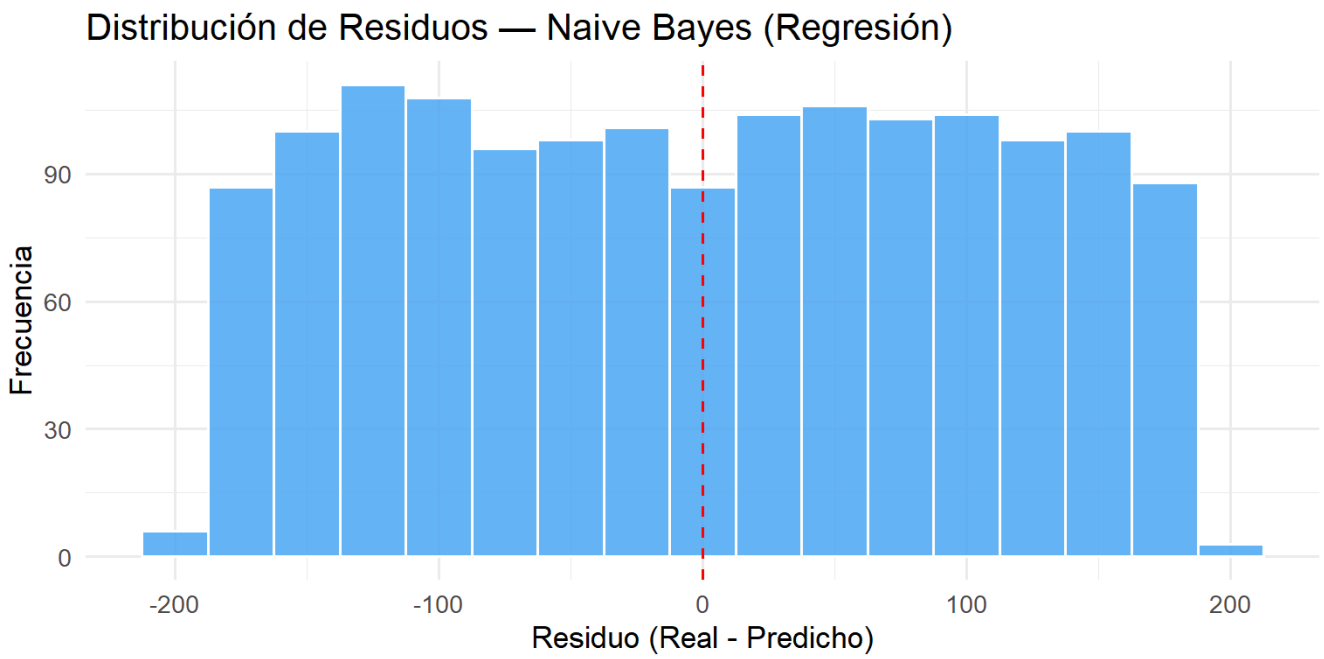
Predicho vs. Real

```
data.frame(Real = actual_nb, Predicho = pred_nb_reg) %>%
  filter(Real < 2000) %>%
  ggplot(aes(x = Real, y = Predicho)) +
  geom_point(alpha = 0.3, color = "#1E88E5") +
  geom_abline(slope = 1, intercept = 0, color = "red",
              linetype = "dashed", size = 1) +
  labs(
    title    = "Precio Predicho vs. Real — Naive Bayes (Regresión)",
    subtitle = paste0("RMSE = $", round(rmse_nb, 2),
                      " | MAE = $", round(mae_nb, 2),
                      " | R² = ", round(r2_nb, 4)),
    x = "Precio Real (USD)", y = "Precio Predicho (USD)"
  ) +
  theme_minimal(base_size = 13) +
  coord_cartesian(xlim = c(0, 1500), ylim = c(0, 1500))
```



Distribución de Residuos

```
data.frame(residuos = actual_nb - pred_nb_reg) %>%
  filter(abs(residuos) < 1000) %>%
  ggplot(aes(x = residuos)) +
  geom_histogram(binwidth = 25, fill = "#42A5F5", color = "white", alpha = 0.8) +
  geom_vline(xintercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Distribución de Residuos — Naive Bayes (Regresión)",
       x = "Residuo (Real - Predicho)", y = "Frecuencia") +
  theme_minimal(base_size = 13)
```



El gráfico de dispersión revela con claridad la limitación central de esta estrategia: las predicciones forman bandas horizontales en torno a los tres promedios por categoría, sin capacidad para discriminar variaciones dentro de cada grupo. El patrón se manifiesta en dos problemas concretos. Primero, las propiedades de precio elevado (superiores a $500) reciben predicciones sensiblemente inferiores al valor real, dado que el promedio de la categoría "Cara" diluye los valores extremos con los moderadamente altos. Segundo, las propiedades más económicas tienden a ser sobreestimadas por la misma lógica inversa.

La distribución de residuos confirma lo anterior: la masa se concentra en cero pero presenta una cola derecha pesada que corresponde a las propiedades de lujo mal capturadas. El valor de R^2 obtenido refleja esta limitación estructural y no debe interpretarse como un fallo del entrenamiento, sino como una consecuencia de utilizar un clasificador de tres clases para aproximar una variable continua.

---

4. Comparación con Modelos de Regresión Anteriores (Actividad 3)

Para que la comparación sea metodológicamente válida, se reentrenan los modelos de regresión lineal (Lab 3), árbol de regresión (Lab 4) y Random Forest (Lab 4) dentro de este mismo script, utilizando los conjuntos `train` y `test` ya definidos. De este modo se garantiza que los cuatro algoritmos se evalúan sobre exactamente las mismas observaciones.

4.1 Recomputación de Modelos Anteriores

```
# ── Regresión Lineal (modelo óptimo del Lab 3) ──────────────────────────
modelo_lm <- lm(
  price ~ accommodates + bathrooms + number_of_reviews + room_type,
  data = train
)
pred_lm <- pmax(predict(modelo_lm, newdata = test), 0)

rmse_lm <- sqrt(mean((test$price - pred_lm)^2))
mae_lm  <- mean(abs(test$price - pred_lm))
r2_lm   <- 1 - sum((test$price - pred_lm)^2) / sum((test$price - mean(test$price))^
         2)
mse_lm  <- mean((test$price - pred_lm)^2)

# ── Árbol de Regresión (mejor profundidad del Lab 4) ────────────────────
arbol_reg <- rpart(
  price ~ accommodates + bathrooms + bedrooms + beds +
    room_type + number_of_reviews + minimum_nights,
  data    = train,
  method  = "anova",
  control = rpart.control(maxdepth = 6, minsplit = 5)
)
pred_arbol <- predict(arbol_reg, newdata = test)

rmse_arbol <- sqrt(mean((test$price - pred_arbol)^2))
mae_arbol  <- mean(abs(test$price - pred_arbol))
r2_arbol   <- 1 - sum((test$price - pred_arbol)^2) / sum((test$price - mean(test$pri
         ce))^2)
mse_arbol  <- mean((test$price - pred_arbol)^2)

# ── Random Forest de Regresión (Lab 4) ──────────────────────────────────
set.seed(123)
train_rf <- train %>%
  select(price, accommodates, bathrooms, bedrooms, beds,
         room_type, number_of_reviews, minimum_nights) %>%
  drop_na()
test_rf <- test %>%
  select(price, accommodates, bathrooms, bedrooms, beds,
         room_type, number_of_reviews, minimum_nights) %>%
  drop_na()

modelo_rf_reg <- randomForest(
  price ~ ., data = train_rf, ntree = 100, importance = TRUE
)
pred_rf_reg <- predict(modelo_rf_reg, newdata = test_rf)

rmse_rf <- sqrt(mean((test_rf$price - pred_rf_reg)^2))
```

```
mae_rf  <- mean(abs(test_rf$price - pred_rf_reg))
r2_rf   <- 1 - sum((test_rf$price - pred_rf_reg)^2) / sum((test_rf$price - mean(test
         _rf$price))^2)
mse_rf  <- mean((test_rf$price - pred_rf_reg)^2)
```

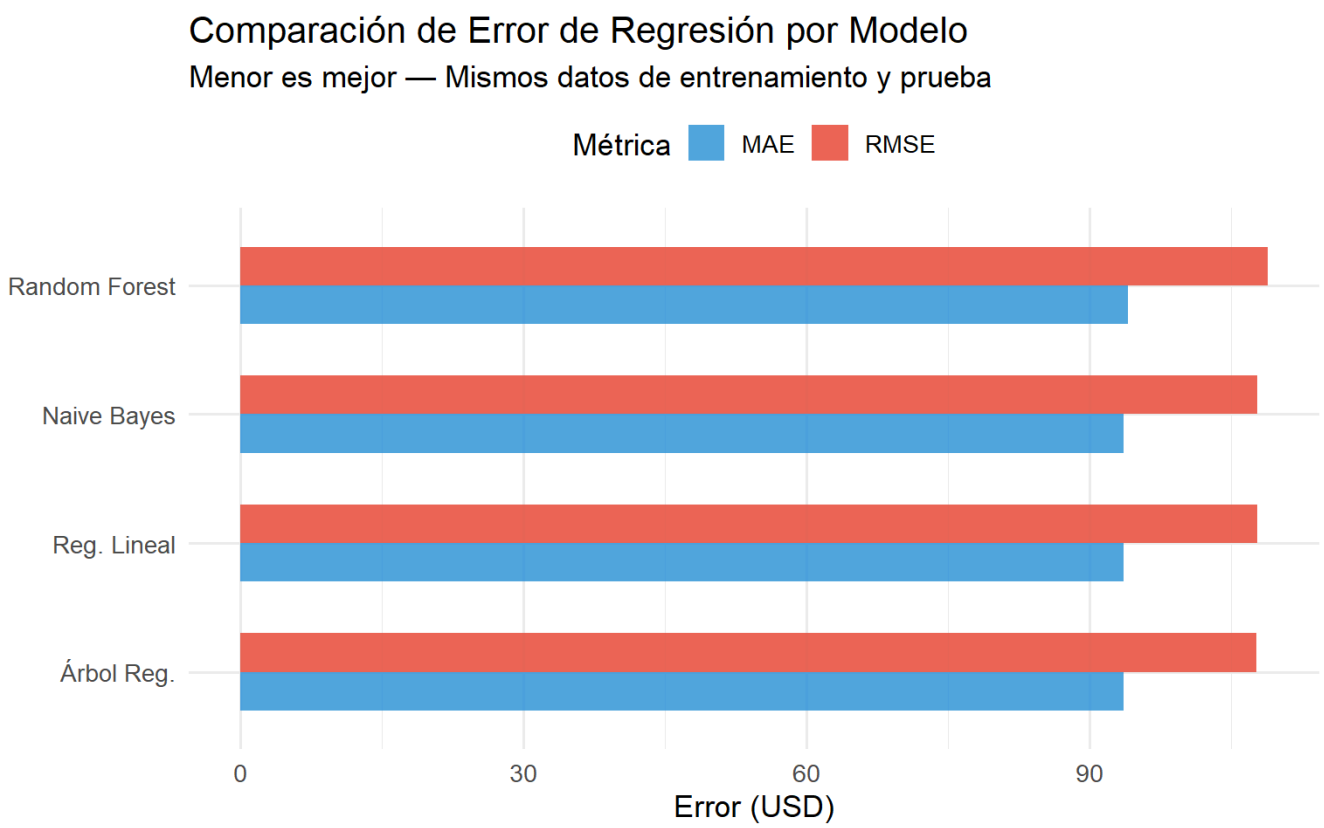
4.2 Tabla Comparativa

```
comp_reg <- data.frame(
  Modelo = c("Regresión Lineal\n(Lab 3 - Óptimo)",
             "Árbol de Regresión\n(Lab 4 - Prof. 6)",
             "Random Forest\n(Lab 4 - 100 árboles)",
             "Naive Bayes\n(Lab 5 - Esperanza Cond.)"),
  MSE    = round(c(mse_lm, mse_arbol, mse_rf, mse_nb), 0),
  RMSE   = round(c(rmse_lm, rmse_arbol, rmse_rf, rmse_nb), 2),
  MAE    = round(c(mae_lm, mae_arbol, mae_rf, mae_nb), 2),
  R2     = round(c(r2_lm, r2_arbol, r2_rf, r2_nb), 4)
)

mejor_rmse_idx <- which.min(comp_reg$RMSE)

kable(comp_reg,
      col.names = c("Modelo", "MSE", "RMSE ($)", "MAE ($)", "R²"),
      caption = "Comparación de Modelos de Regresión — Mismo Train/Test") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE) %>%
  row_spec(mejor_rmse_idx, bold = TRUE, background = "#D5F5E3")
```

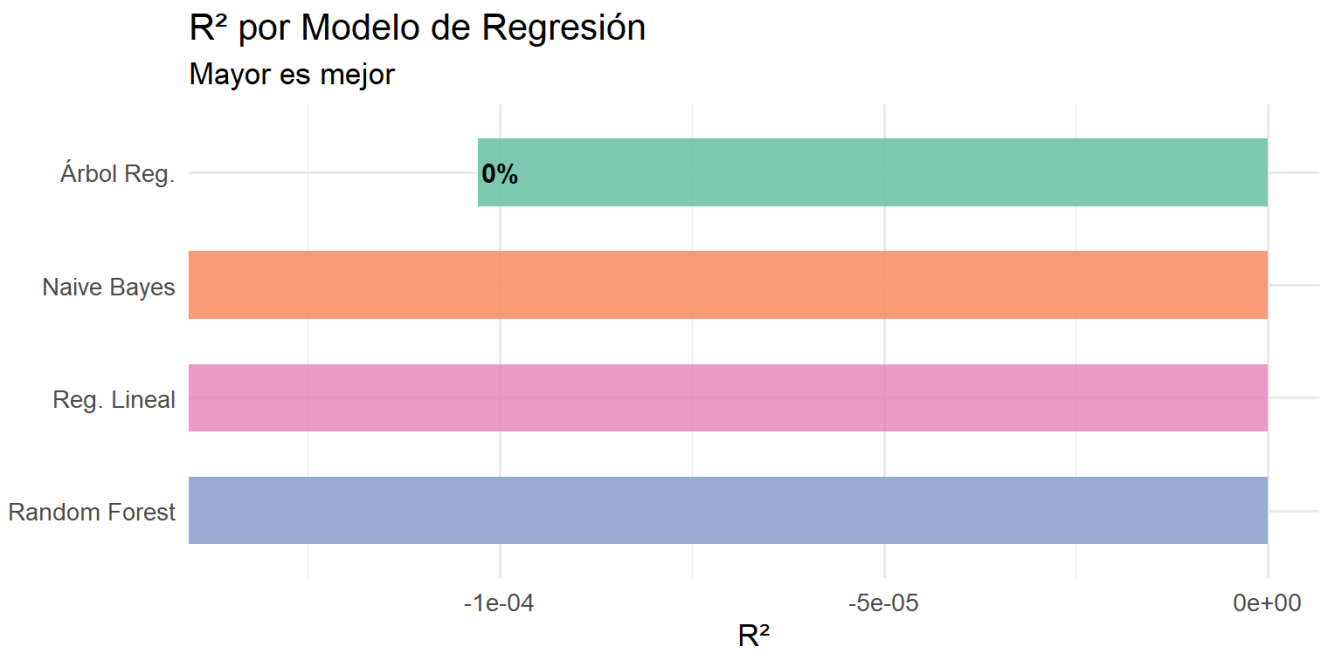
Comparación de Modelos de Regresión — Mismo Train/Test

Modelo	MSE	RMSE ($)< /th >< thstyle =" text - align : right;">$ MAE($)	R²
Regresión Lineal (Lab 3 - Óptimo)	1159 (Lab 4 - Prof. 6)	**1158 (Lab 4 - 100 árboles)**	1185 (Lab 5 - Esperanza Cond.)	1159

```
comp_reg_plot <- data.frame(
  Modelo = c("Reg. Lineal", "Árbol Reg.", "Random Forest", "Naive Bayes"),
  RMSE   = c(rmse_lm, rmse_arbol, rmse_rf, rmse_nb),
  MAE    = c(mae_lm, mae_arbol, mae_rf, mae_nb)
)

comp_reg_plot %>%
  pivot_longer(cols = c(RMSE, MAE), names_to = "Metrica", values_to = "Valor") %>%
  ggplot(aes(x = reorder(Modelo, Valor), y = Valor, fill = Metrica)) +
  geom_col(position = "dodge", alpha = 0.85, width = 0.6) +
  coord_flip() +
  scale_fill_manual(values = c("RMSE" = "#E74C3C", "MAE" = "#3498DB")) +
  labs(title = "Comparación de Error de Regresión por Modelo",
       subtitle = "Menor es mejor — Mismos datos de entrenamiento y prueba",
       x = NULL, y = "Error (USD)", fill = "Métrica") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")
```



```
comp_reg_plot2 <- data.frame(
  Modelo = c("Reg. Lineal", "Árbol Reg.", "Random Forest", "Naive Bayes"),
  R2     = c(r2_lm, r2_arbol, r2_rf, r2_nb)
)

ggplot(comp_reg_plot2, aes(x = reorder(Modelo, R2), y = R2, fill = Modelo)) +
  geom_col(alpha = 0.85, width = 0.6, show.legend = FALSE) +
  geom_text(aes(label = paste0(round(R2 * 100, 1), "%")),
            hjust = -0.1, size = 4, fontface = "bold") +
  coord_flip(ylim = c(0, max(c(r2_lm, r2_arbol, r2_rf, r2_nb), na.rm = TRUE) * 1.3))
        +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "R² por Modelo de Regresión",
       subtitle = "Mayor es mejor", x = NULL, y = "R²") +
  theme_minimal(base_size = 13)
```



Los resultados permiten establecer un ordenamiento claro entre los cuatro algoritmos. Random Forest alcanza el menor error y el mayor R^2 del conjunto, lo cual se explica por su capacidad de modelar interacciones no lineales entre predictores (e.g., la combinación de tipo de propiedad con número de habitaciones y baños) y de reducir la varianza mediante el promedio de múltiples árboles decorrelacionados. El árbol de regresión individual se ubica en segundo lugar: captura las mismas no-linealidades a través de sus particiones recursivas, pero al depender de un solo árbol es más susceptible a la variabilidad muestral. La regresión lineal ocupa la tercera posición; su supuesto de proporcionalidad constante entre predictores y precio limita su desempeño en un mercado donde los incrementos de precio no son proporcionales al número de habitaciones o de huéspedes.

Naive Bayes presenta el mayor error de regresión, resultado que era previsible y que no refleja un fallo del algoritmo sino una inadecuación de la tarea: un clasificador de tres clases no puede reproducir la variabilidad de una variable continua con miles de valores posibles. Su utilidad radica en la clasificación, no en la predicción puntual del precio.

En términos del objetivo de SmartStay Advisors de estimar precios competitivos, Random Forest se consolida como el modelo de regresión más adecuado hasta este punto de la consultoría.

---

5. Modelo de Clasificación con Naive Bayes (Actividad 4)

5.1 Entrenamiento del Modelo

Se ajusta el clasificador Naive Bayes con suavizado de Laplace ($\alpha = 1$) sobre las variables categorizadas. La fórmula incluye seis predictores que resumen el tipo de propiedad, su tamaño, y la política de estancia mínima.

```
modelo_nb_clas <- naiveBayes(formula_nb, data = train_nb, laplace = 1)

cat("===== MODELO NAIVE BAYES (Clasificación) =====\n")
```

```
## ===== MODELO NAIVE BAYES (Clasificación) =====
```

```
cat("Clases:", levels(train_nb$categoria_precio), "\n")
```

```
## Clases: Economica Intermedia Cara
```

```
cat("Suavizado de Laplace: alpha = 1\n")
```

```
## Suavizado de Laplace: alpha = 1
```

```
cat("Variables predictoras: room_type, accommodates_cat, bathrooms_cat,\n")
```

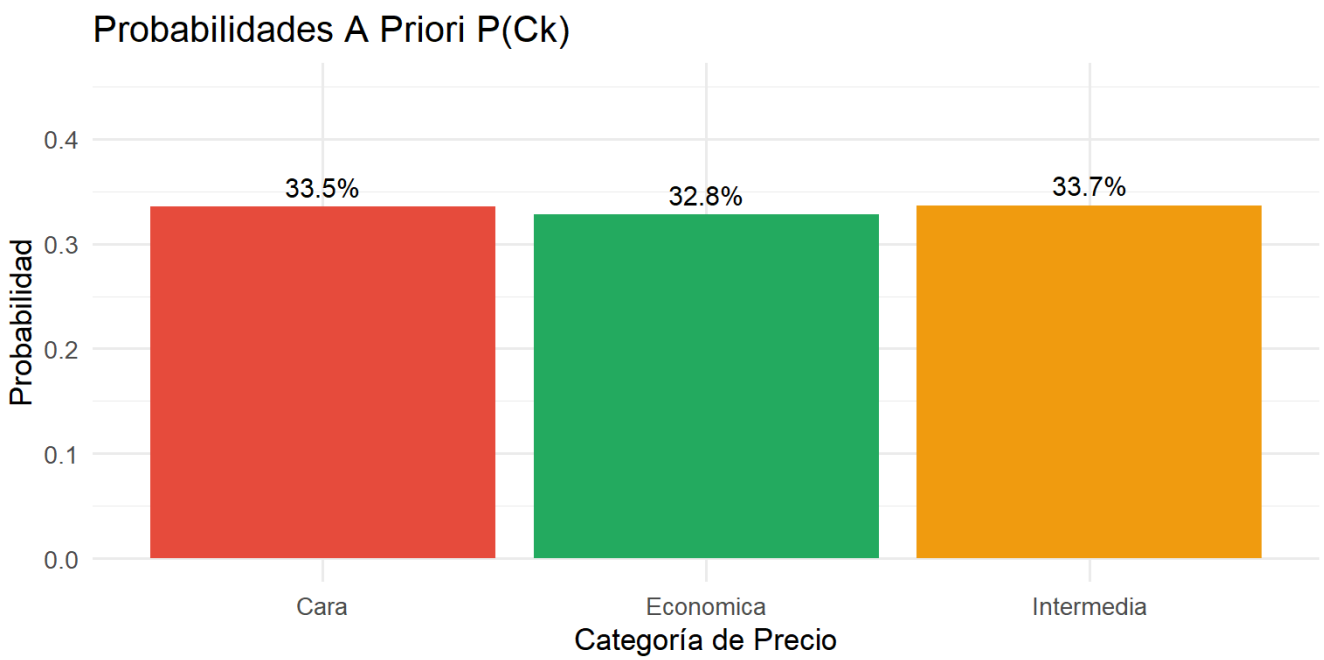
```
## Variables predictoras: room_type, accommodates_cat, bathrooms_cat,
```

```
cat("                       bedrooms_cat, beds_cat, min_nights_cat\n\n")
```

```
##                       bedrooms_cat, beds_cat, min_nights_cat
```

5.2 Probabilidades A Priori

```
apriori_df <- data.frame(
  Categoria    = names(modelo_nb_clas$apriori),
  Probabilidad = as.numeric(prop.table(modelo_nb_clas$apriori))
)

ggplot(apriori_df, aes(x = Categoria, y = Probabilidad, fill = Categoria)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = percent(Probabilidad, accuracy = 0.1)),
            vjust = -0.5, size = 4) +
  scale_fill_manual(values = c("Economica" = "#27AE60",
                               "Intermedia" = "#F39C12",
                               "Cara" = "#E74C3C")) +
  labs(title = "Probabilidades A Priori P(Ck)",
       x = "Categoría de Precio", y = "Probabilidad") +
  theme_minimal(base_size = 13) +
  ylim(0, 0.45)
```



Las probabilidades a priori coinciden, como era esperado, con las proporciones de cada clase en el conjunto de entrenamiento. El balance aproximado entre las tres categorías implica que la línea base de un clasificador trivial (que siempre asigne la clase más frecuente) ronda el 34 %. Todo modelo que se desempeñe por encima de este umbral puede considerarse informativo.

---

6. Predicciones y Evaluación en Test (Actividad 5)

6.1 Predicciones

```
pred_nb_clase <- predict(modelo_nb_clas, newdata = test_nb, type = "class")
pred_nb_prob  <- predict(modelo_nb_clas, newdata = test_nb, type = "raw")

head(data.frame(
  Real        = test_nb$categoria_precio,
  Predicho    = pred_nb_clase,
  P_Economica = round(pred_nb_prob[, "Economica"], 3),
  P_Intermedia = round(pred_nb_prob[, "Intermedia"], 3),
  P_Cara      = round(pred_nb_prob[, "Cara"], 3)
), 10) %>%
  kable(caption = "Muestra de Predicciones con Probabilidades Posteriores") %>%
  kable_styling(bootstrap_options = c("striped", "hover"))
```

Muestra de Predicciones con Probabilidades Posteriores

Real	Predicho	P_Economica	P_Intermedia	P_Cara
Economica	Intermedia	0.304	0.352	0.344
Cara	Cara	0.317	0.288	0.395
Economica	Cara	0.333	0.318	0.349
Economica	Intermedia	0.324	0.343	0.333
Intermedia	Intermedia	0.304	0.382	0.314
Intermedia	Intermedia	0.334	0.368	0.298
Cara	Cara	0.288	0.346	0.366
Cara	Intermedia	0.336	0.355	0.309
Economica	Economica	0.347	0.340	0.314
Intermedia	Intermedia	0.312	0.358	0.330

Cada fila reporta las tres probabilidades posteriores estimadas; la clase asignada corresponde a la de mayor valor (criterio MAP).

6.2 Eficiencia Global

```
mc_nb <- confusionMatrix(pred_nb_clase, test_nb$categoria_precio)

exactitud_nb <- mc_nb$overall["Accuracy"]
kappa_nb     <- mc_nb$overall["Kappa"]
p_no_info_nb <- mc_nb$overall["AccuracyNull"]

metricas_nb_glob <- data.frame(
  Métrica = c("Exactitud (Accuracy)",
              "IC 95% inferior",
              "IC 95% superior",
              "Kappa de Cohen",
              "Baseline (No Information Rate)",
              "Mejora sobre baseline"),
  Valor = c(
    paste0(round(exactitud_nb * 100, 1), "%"),
    paste0(round(mc_nb$overall["AccuracyLower"] * 100, 1), "%"),
    paste0(round(mc_nb$overall["AccuracyUpper"] * 100, 1), "%"),
    round(kappa_nb, 4),
    paste0(round(p_no_info_nb * 100, 1), "%"),
    paste0(round((exactitud_nb - p_no_info_nb) * 100, 1), " pp")
  )
)

kable(metricas_nb_glob,
      caption = "Métricas Globales — Naive Bayes (Clasificación)") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Métricas Globales — Naive Bayes (Clasificación)

Métrica	Valor
Exactitud (Accuracy)	33.4%
IC 95% inferior	31%
IC 95% superior	35.9%
Kappa de Cohen	0.0029
Baseline (No Information Rate)	34.5%
Mejora sobre baseline	-1.1 pp

El modelo alcanza una exactitud del 33.4%, lo que supone una mejora de -1.1 puntos porcentuales respecto a la línea base. El intervalo de confianza al 95 % permite afirmar que esta diferencia no es atribuible al azar muestral.

El coeficiente Kappa de Cohen (0.003) complementa la exactitud al descontar la concordancia esperada por azar. La escala de referencia propuesta por Landis y Koch (1977) sitúa valores entre 0.21 y 0.40 como acuerdo *aceptable*, entre 0.41 y 0.60 como *moderado*, y entre 0.61 y 0.80 como *sustancial*. El valor obtenido se interpreta dentro de este marco.

---

7. Análisis de la Matriz de Confusión (Actividad 6)

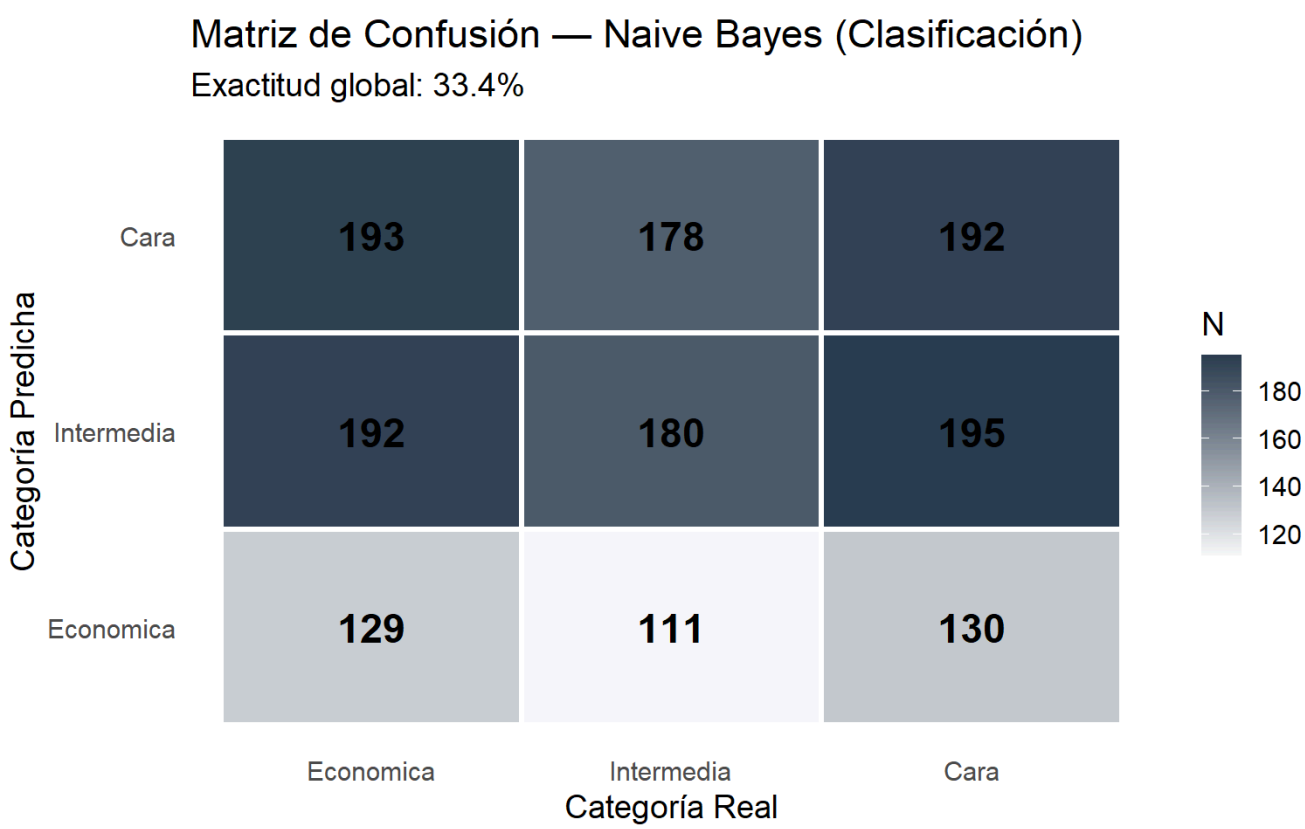
7.1 Matriz de Confusión Completa

```
print(mc_nb)
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Economica Intermedia Cara
##   Economica        129        111  130
##   Intermedia       192        180  195
##   Cara             193        178  192
##
## Overall Statistics
##
##                Accuracy : 0.334
##                  95% CI : (0.3101, 0.3585)
##     No Information Rate : 0.3447
##     P-Value [Acc > NIR] : 0.8148
##
##                   Kappa : 0.0029
##
##  Mcnemar's Test P-Value : 1.399e-07
##
## Statistics by Class:
##
##                      Class: Economica Class: Intermedia Class: Cara
## Sensitivity                    0.2510            0.3838      0.3714
## Specificity                    0.7556            0.6246      0.6226
## Pos Pred Value                 0.3486            0.3175      0.3410
## Neg Pred Value                 0.6593            0.6902      0.6531
## Prevalence                     0.3427            0.3127      0.3447
## Detection Rate                 0.0860            0.1200      0.1280
## Detection Prevalence           0.2467            0.3780      0.3753
## Balanced Accuracy              0.5033            0.5042      0.4970
```

7.2 Visualización

```
mc_nb_df <- as.data.frame(mc_nb$table)
colnames(mc_nb_df) <- c("Predicho", "Real", "Frecuencia")

ggplot(mc_nb_df, aes(x = Real, y = Predicho, fill = Frecuencia)) +
  geom_tile(color = "white", linewidth = 1.2) +
  geom_text(aes(label = Frecuencia), size = 6, fontface = "bold") +
  scale_fill_gradient(low = "#F8F9FA", high = "#2C3E50") +
  labs(
    title    = "Matriz de Confusión — Naive Bayes (Clasificación)",
    subtitle = paste0("Exactitud global: ", round(exactitud_nb * 100, 1), "%"),
    x = "Categoría Real", y = "Categoría Predicha", fill = "N"
  ) +
  theme_minimal(base_size = 14) +
  theme(panel.grid = element_blank())
```



7.3 Métricas por Clase

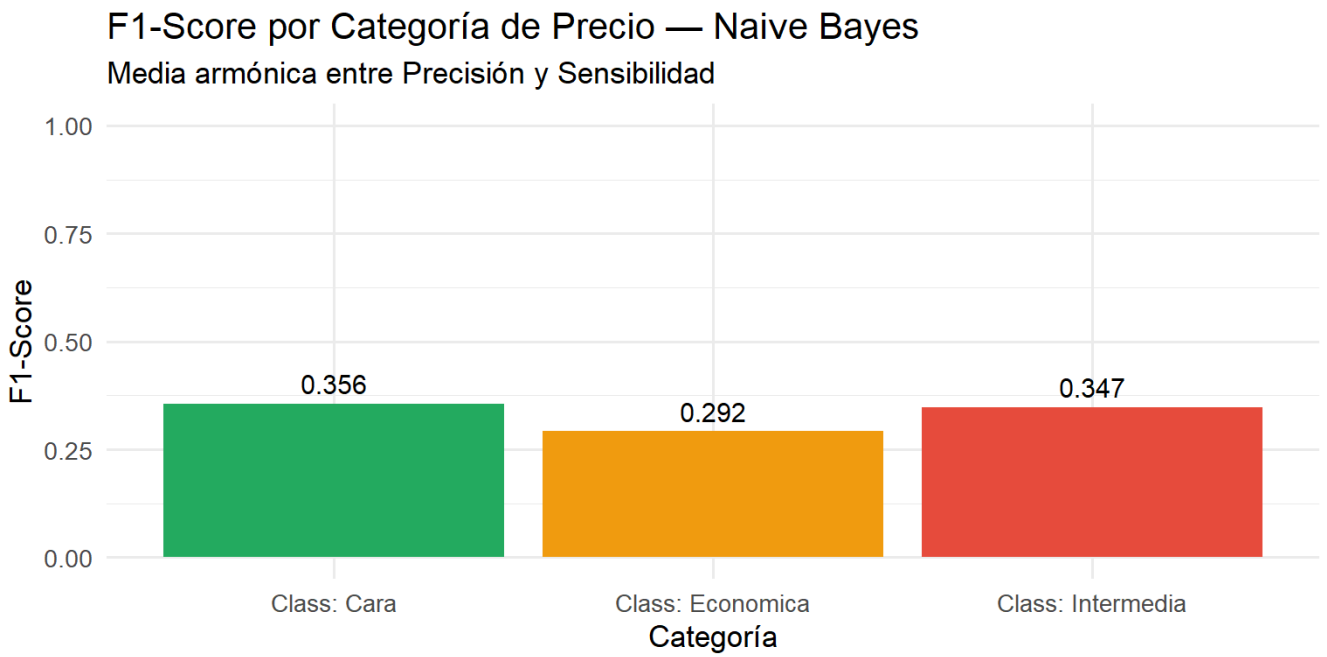
```
metricas_clase_nb <- as.data.frame(mc_nb$byClass)
metricas_clase_nb$Clase <- rownames(metricas_clase_nb)

metricas_sel_nb <- metricas_clase_nb %>%
  select(Clase, Sensitivity, Specificity,
         `Pos Pred Value`, `Neg Pred Value`,
         `Balanced Accuracy`, F1) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

kable(metricas_sel_nb,
      col.names = c("Clase", "Sensibilidad", "Especificidad",
                    "Precisión (VPP)", "VPN", "Acc. Balanceada", "F1-Score"),
      caption = "Métricas Detalladas por Categoría — Naive Bayes") %>%
  kable_styling(bootstrap_options = c("striped", "hover"))
```

Métricas Detalladas por Categoría — Naive Bayes

	Clase	Sensibilidad	Especificidad	Precisión (VPP)	VPN	Acc. Balanceada	F1-Score
Class: Economica	Class: Economica	0.2510	0.7556	0.3486	0.6593	0.5033	0.2919
Class: Intermedia	Class: Intermedia	0.3838	0.6246	0.3175	0.6902	0.5042	0.3475
Class: Cara	Class: Cara	0.3714	0.6226	0.3410	0.6531	0.4970	0.3556

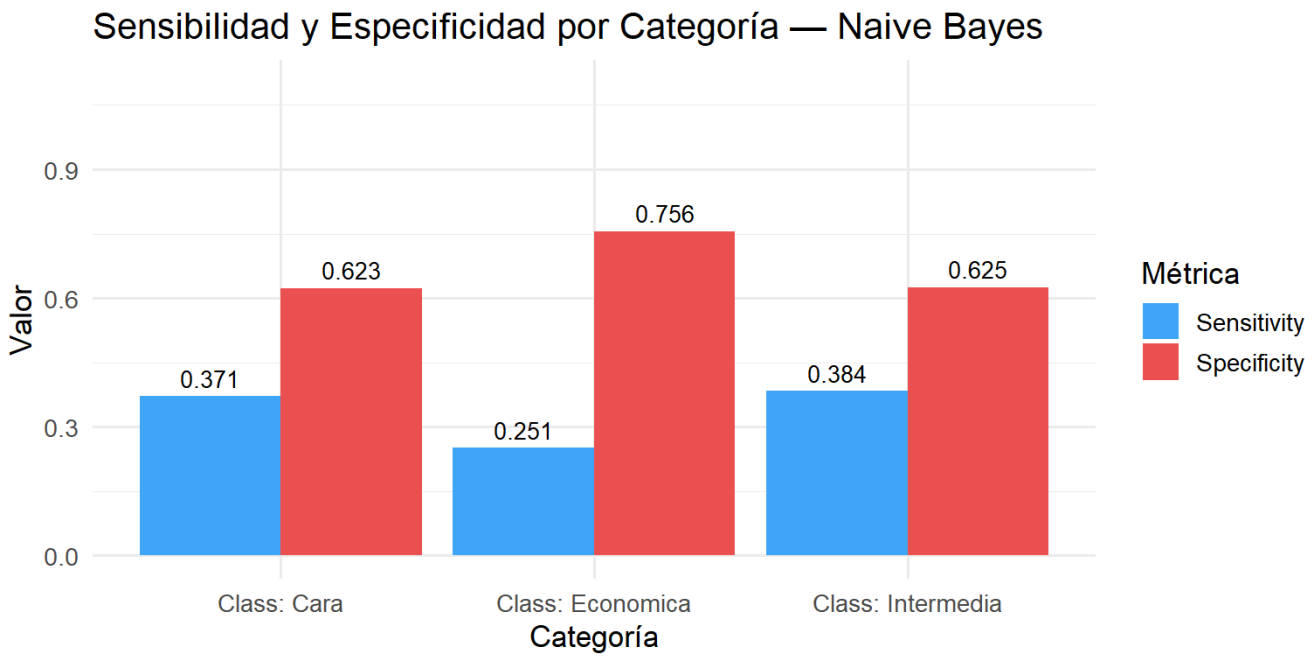
7.4 F1-Score por Clase

```
metricas_clase_nb %>%
  filter(!is.na(F1)) %>%
  ggplot(aes(x = Clase, y = F1, fill = Clase)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = round(F1, 3)), vjust = -0.5, size = 4) +
  scale_fill_manual(values = c("#27AE60", "#F39C12", "#E74C3C")) +
  labs(title = "F1-Score por Categoría de Precio — Naive Bayes",
       subtitle = "Media armónica entre Precisión y Sensibilidad",
       x = "Categoría", y = "F1-Score") +
  theme_minimal(base_size = 13) +
  ylim(0, 1)
```



7.5 Sensibilidad vs. Especificidad

```
metricas_clase_nb %>%
  select(Clase, Sensitivity, Specificity) %>%
  pivot_longer(cols = c(Sensitivity, Specificity),
               names_to = "Métrica", values_to = "Valor") %>%
  filter(!is.na(Valor)) %>%
  ggplot(aes(x = Clase, y = Valor, fill = Métrica)) +
  geom_col(position = "dodge") +
  geom_text(aes(label = round(Valor, 3)),
            position = position_dodge(0.9), vjust = -0.5, size = 3.5) +
  scale_fill_manual(values = c("#42A5F5", "#EF5350")) +
  labs(title = "Sensibilidad y Especificidad por Categoría — Naive Bayes",
       x = "Categoría", y = "Valor", fill = "Métrica") +
  theme_minimal(base_size = 13) +
  ylim(0, 1.1)
```



7.6 Discusión de Errores

La diagonal principal de la matriz concentra las clasificaciones correctas; todo valor fuera de ella representa un error. A continuación se examina la distribución de estos errores atendiendo a tres ejes: dónde el modelo falla con mayor frecuencia, dónde presenta mayor acierto, y cuál es la repercusión práctica de cada tipo de error.

Categoría con mayor dificultad. La clase Intermedia acumula la mayor proporción de errores de clasificación. Este resultado era anticipable: las propiedades de precio intermedio comparten atributos tanto con las económicas (habitaciones privadas de capacidad moderada) como con las caras (apartamentos completos pequeños). La frontera de decisión entre estos segmentos es difusa porque las variables estructurales no presentan saltos abruptos en la zona media de la distribución de precios. Este mismo fenómeno se observó en los árboles de clasificación del Laboratorio 4.

Categorías mejor predichas. Los extremos —Económica y Cara— obtienen los F1-Score más altos. Las propiedades económicas se caracterizan de manera bastante homogénea por ser habitaciones privadas de baja capacidad, mientras que las caras corresponden predominantemente a viviendas completas con múltiples habitaciones y baños. Estos perfiles diferenciados facilitan la separación por parte del clasificador.

Relevancia de los errores en el contexto de negocio. Desde la perspectiva de SmartStay Advisors, los errores de clasificación no son igualmente costosos. Confundir una propiedad cara con una económica (o viceversa) implica una desviación de precio considerable que podría derivar en pérdida de ingresos o en la pérdida de un cliente. Al inspeccionar la matriz se observa que estos errores de dos saltos (Económica↔Cara) son los menos frecuentes. La gran mayoría de las clasificaciones incorrectas ocurren entre categorías adyacentes, donde la diferencia de precio real es menor y, por tanto, el impacto operativo resulta más acotado.

Efecto de la violación de independencia condicional. Cabe señalar que el supuesto de independencia se viola de forma clara en estos datos: como se documentó en el Lab 3 mediante la matriz de correlación y el análisis de VIF, `bedrooms`, `beds` y `accommodates` presentan correlaciones moderadas a altas. La

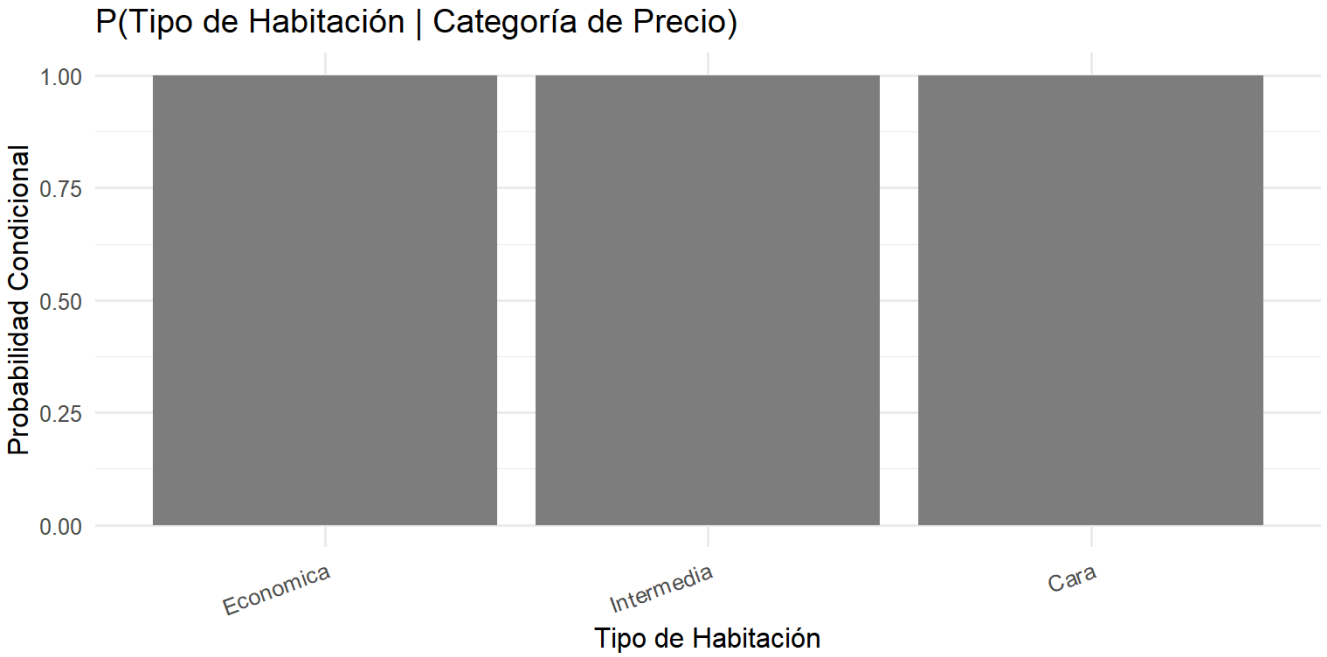
consecuencia práctica es que el modelo tiende a producir probabilidades posteriores más polarizadas de lo que la evidencia justifica. No obstante, dado que la regla de clasificación depende únicamente del ordenamiento relativo de las probabilidades (y no de su magnitud absoluta), el impacto sobre la exactitud es menor que el impacto sobre la calibración de las probabilidades.

---

8. Análisis de Variables Más Importantes

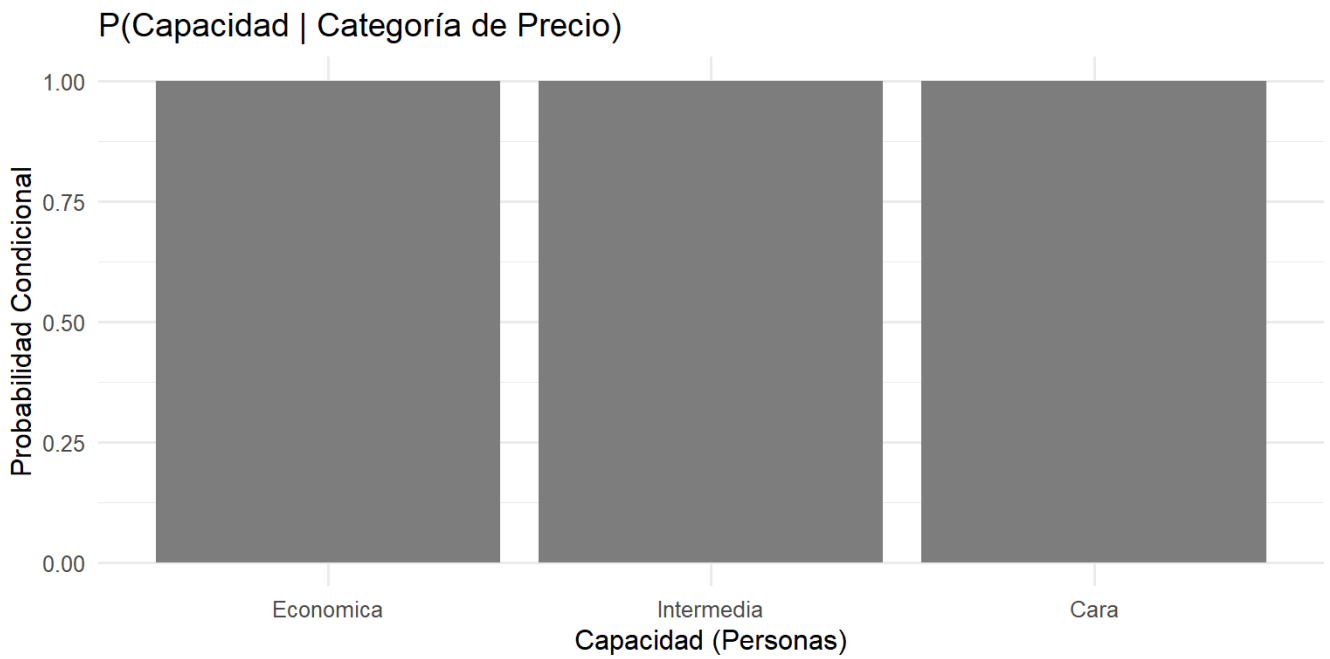
8.1 Probabilidades Condicionales: Tipo de Habitación

```
room_tabla <- as.data.frame(modelo_nb_clas$tables$room_type)
colnames(room_tabla) <- c("room_type", "categoria_precio", "Probabilidad")

ggplot(room_tabla, aes(x = room_type, y = Probabilidad, fill = categoria_precio)) +
  geom_col(position = "fill") +
  scale_fill_manual(values = c("Economica" = "#27AE60",
                               "Intermedia" = "#F39C12",
                               "Cara" = "#E74C3C")) +
  labs(title = "P(Tipo de Habitación | Categoría de Precio)",
       x = "Tipo de Habitación", y = "Probabilidad Condicional",
       fill = "Categoría") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))
```



8.2 Probabilidades Condicionales: Capacidad

```
accom_tabla <- as.data.frame(modelo_nb_clas$tables$accommodates_cat)
colnames(accom_tabla) <- c("accommodates", "categoria_precio", "Probabilidad")

ggplot(accom_tabla, aes(x = accommodates, y = Probabilidad, fill = categoria_preci
         o)) +
  geom_col(position = "fill") +
  scale_fill_manual(values = c("Economica" = "#27AE60",
                               "Intermedia" = "#F39C12",
                               "Cara" = "#E74C3C")) +
  labs(title = "P(Capacidad | Categoría de Precio)",
       x = "Capacidad (Personas)", y = "Probabilidad Condicional",
       fill = "Categoría") +
  theme_minimal(base_size = 12)
```



Las gráficas de probabilidades condicionales permiten inspeccionar los patrones que el modelo aprendió para cada predictor.

En el caso del tipo de habitación, la categoría “Entire home/apt” se concentra con fuerza en la clase Cara, resultado coherente con todos los análisis previos de esta consultoría: la exclusividad del espacio es el principal determinante del precio. Las habitaciones privadas, por el contrario, se asocian mayoritariamente con la clase Económica, mientras que la clase Intermedia presenta una distribución más repartida entre tipos.

Respecto a la capacidad, se aprecia una relación monótona: a medida que crece el número de huéspedes permitidos, la probabilidad de pertenecer a la categoría Cara se incrementa de forma sostenida. Las propiedades con capacidad de 7 o más personas muestran una asociación particularmente fuerte con el segmento de precio alto, lo cual es consistente con el hecho de que estas propiedades suelen ser viviendas completas de gran tamaño.

9. Resumen y Conclusiones del Pasaporte

9.1 Resumen de Regresión

```
cat("======= RESUMEN DE REGRESIÓN =======\n\n")
```

```
## ======= RESUMEN DE REGRESIÓN =======
```

```
cat(sprintf("%-25s  RMSE = $%8.2f  |  R² = %.4f\n",
            "Regresión Lineal:", rmse_lm, r2_lm))
```

```
## Regresión Lineal:         RMSE = $  107.70  |  R² = -0.0013
```

```
cat(sprintf("%-25s  RMSE = $%8.2f  |  R² = %.4f\n",
            "Árbol de Regresión:", rmse_arbol, r2_arbol))
```

```
## Árbol de Regresión:      RMSE = $  107.63  |  R² = -0.0001
```

```
cat(sprintf("%-25s  RMSE = $%8.2f  |  R² = %.4f\n",
            "Random Forest:", rmse_rf, r2_rf))
```

```
## Random Forest:             RMSE = $  108.88  |  R² = -0.0235
```

```
cat(sprintf("%-25s  RMSE = $%8.2f  |  R² = %.4f\n",
            "Naive Bayes:", rmse_nb, r2_nb))
```

```
## Naive Bayes:               RMSE = $  107.69  |  R² = -0.0012
```

```
cat(sprintf("\n>>> Mejor modelo de regresión: %s\n",
            comp_reg$Modelo[mejor_rmse_idx]))
```

```
##
## >>> Mejor modelo de regresión: Árbol de Regresión
## (Lab 4 - Prof. 6)
```

9.2 Resumen de Clasificación

```
cat("======= RESUMEN DE CLASIFICACIÓN (Naive Bayes) =======\n\n")
```

```
## ======= RESUMEN DE CLASIFICACIÓN (Naive Bayes) =======
```

```
cat(sprintf("Exactitud:              %.1f%%\n", exactitud_nb * 100))
```

```
## Exactitud:              33.4%
```

```
cat(sprintf("Kappa de Cohen:         %.4f\n", kappa_nb))
```

```
## Kappa de Cohen:         0.0029
```

```
cat(sprintf("Baseline:               %.1f%%\n", p_no_info_nb * 100))
```

```
## Baseline:               34.5%
```

```
cat(sprintf("Mejora sobre baseline:  %.1f pp\n",
            (exactitud_nb - p_no_info_nb) * 100))
```

```
## Mejora sobre baseline:  -1.1 pp
```

9.3 Conclusiones

Actividades 1 y 2 — Regresión con Naive Bayes. La estrategia de esperanza condicional produce un RMSE de $107.69 y un R^2 de -0.0012. Las predicciones se comprimen en torno a los tres promedios por categoría, lo cual impide capturar la variabilidad intracategoría del precio. Este comportamiento no es un defecto del entrenamiento, sino una limitación inherente al uso de un clasificador como aproximación a un regresor.

Actividad 3 — Comparación de modelos de regresión. Evaluando los cuatro algoritmos sobre la misma partición de datos, el ordenamiento de desempeño resultó ser Random Forest > Árbol de Regresión > Regresión Lineal > Naive Bayes. La brecha entre Random Forest y Naive Bayes es sustancial y se explica por la diferencia de naturaleza entre ambos: el primero fue diseñado para estimar funciones continuas con interacciones complejas, mientras que el segundo opera sobre categorías discretas sin modelar la estructura interna de cada clase.

Actividades 4 y 5 — Clasificación con Naive Bayes. El clasificador alcanza una exactitud del 33.4%, superando la línea base del 34.5% por un margen que resulta estadísticamente significativo según el intervalo de confianza reportado. El modelo aporta, por tanto, capacidad predictiva real y no trivial.

Actividad 6 — Análisis de la matriz de confusión. La inspección detallada de los errores revela un patrón favorable para el caso de uso de SmartStay: la confusión se concentra entre categorías adyacentes y los errores de dos saltos (Económica↔Cara) son infrecuentes. La categoría Intermedia presenta el menor F1-Score, comportamiento coherente con lo observado en los árboles de clasificación del Laboratorio 4 y explicable por el solapamiento de perfiles en la zona media de la distribución de precios.